PRCL-0019 — Explained

A plain-language walkthrough of the Sales Effectiveness / Lead Category Prediction project

Project	PRCL-0019: Sales Effectiveness — Lead Category Prediction Using Machine Learning
Client	FicZon Inc. (IT Solutions Provider)
Dataset	7,422 lead records, 9 columns, pulled live from a MySQL database
Goal	Predict whether a new lead is High Potential or Low Potential at the moment it is captured
Winning Model	Gradient Boosting Classifier — 72.04% test accuracy, smallest overfitting gap of all six models tried

1. What Problem Was This Solving?

FicZon Inc. sells IT products and picks up most of its leads through its website and call channels. Sales teams were sorting incoming leads into "good" vs. "bad" by hand, and the quality-control process that checks this only kicks in after the fact — it tells the team what went wrong last month, not which lead to call first today.

The project's job was to replace that guesswork with a machine learning model that scores every lead the moment it comes in, tagging it High Potential or Low Potential so agents know where to focus their time immediately, instead of finding out weeks later.

2. Where the Data Came From

Unlike a typical training exercise that starts from a downloaded spreadsheet, this project connected directly to FicZon's live MySQL database and pulled 7,422 real lead records covering 9 fields: when the lead was created, which product it was for, how it arrived (Source), contact details, the assigned sales agent, location, delivery mode, and its current status.

Because it's a live client database, the password was never written into the code — it was typed in securely each session using Python's getpass() tool, and a untouched backup copy of the raw pull was kept aside before any cleaning began.

3. What the Exploration Showed

- Call and Website are by far the two biggest lead sources, matching FicZon's own description of itself as a digitally-driven business.
- Workload is lopsided: a few sales agents (especially one nicknamed Sales-Agent-4) handle far more leads than the rest of the 13-person team.
- Bangalore and Chennai dominate the location breakdown, with a long tail of smaller cities and an "Other Locations" bucket.
- The Status field — the raw column everything else is built from — had 11 different values, with Junk Lead and Not Responding as the two most common outcomes.

That last point turned out to matter a lot, as the next section explains.

4. Cleaning the Data

A few columns (Product_ID, Source, Mobile, Sales_Agent, Location) had missing or placeholder entries — things like blank text or the literal string "#VALUE!" standing in for a missing phone number. These were filled in with an explicit "Unknown" label rather than deleted, so the fact that something was missing stayed useful information for the model rather than being thrown away. Two exact duplicate rows were removed, and a handful of unusual Product_ID values were capped (winsorized) instead of dropped, so no lead record was lost purely because one field looked odd.

5. The Target Variable — and the Bug That Nearly Broke Everything

The whole project depends on a column called Lead_Category (High Potential vs. Low Potential), but the raw table doesn't have one — it has to be built from the Status field. This is the most important part of the story.

5.1 What went wrong

The first attempt at building this mapping used a dictionary of expected status labels like 'Converted', 'Qualified', and 'In Progress'. The problem: Python treats text matching as case-sensitive, and the real values sitting in the database were spelled differently — 'CONVERTED' and 'converted' (two separate values!) instead of 'Converted', and 'In Progress Positive' / 'In Progress Negative' instead of a single 'In Progress'. Almost none of the guessed labels matched the real ones.

The result was quiet but serious: nearly every lead fell through to a default bucket and ended up in a single class. There was no error message at the moment the bug was introduced — the code ran fine. It only surfaced downstream, when some models refused to train because they were being asked to learn from data that had only one possible answer, and other models reported a suspicious, meaningless 100% accuracy (which is trivially easy when there's only one class to predict).

5.2 How it was fixed

The fix was to stop guessing and inspect the real, exact status text living in the database, then build the mapping from those verified strings. The corrected version groups CONVERTED, converted, In Progress Positive, Potential, and Long Term into High Potential, and groups In Progress Negative, Not Responding, Just Enquiry, Junk Lead, LOST, and Open into Low Potential. Every one of the 11 real status values ended up mapped, with none left over — confirmed by checking for unmapped values before moving on.

After the fix, the split came out to a realistic 61.6% Low Potential vs. 38.4% High Potential (4,571 vs. 2,849 leads) — a genuine two-class target the models could actually learn from.

6. Preparing the Data for Modelling

A few extra features were engineered to give the model more to work with: the lead's creation date was split into year, month, and day (to catch seasonal patterns); the phone/email fields were converted into simple yes/no flags for whether contact details were provided at all, rather than using the raw numbers or addresses (which carry no predictive meaning on their own); and the Status column itself was dropped once Lead_Category was derived from it, to stop it from leaking the answer directly into the model's inputs.

Categorical columns were one-hot encoded, which expanded the feature set to 64 columns (mostly because Sales_Agent and Location have many distinct values). The data was then split into 5,936 training rows and 1,484 held-out test rows.

7. Building and Comparing the Models

Five algorithms were trained and compared — Logistic Regression, Decision Tree, Random Forest, K-Nearest Neighbours, and Gradient Boosting — each deliberately tuned toward simpler, more conservative settings than their defaults, because 64 mostly-sparse one-hot columns on a moderate-sized dataset is exactly the situation where models tend to memorize noise instead of learning real patterns.

Random Forest was additionally put through a formal hyperparameter search (GridSearchCV), which improved its ability to catch High Potential leads but, as the next section shows, came at a cost.

Model	Train Acc.	Test Acc.	F1-Score	Train–Test Gap

Gradient Boosting	74.73%	72.04%	0.7112	2.70 pts
KNN	74.24%	69.00%	0.6810	5.24 pts
Logistic Regression	70.84%	68.13%	0.6851	2.71 pts
Random Forest (Tuned)	74.41%	67.72%	0.6816	6.69 pts
Random Forest	71.31%	66.31%	0.6676	5.00 pts
Decision Tree	67.05%	63.68%	0.6402	3.37 pts

Gradient Boosting came out ahead in every column that matters: highest test accuracy, highest F1-score, and — critically — the smallest gap between how well it did on training data versus unseen test data. A small gap is the signal that a model is generalizing to new leads rather than just memorizing old ones.

The tuned Random Forest is a good illustration of a trap: grid search improved its cross-validated training score, but its train-test gap nearly tripled compared to the untuned version, meaning the tuning made it better at fitting the data it had already seen without making it meaningfully better at predicting new leads.

8. What Drives the Predictions

Looking inside the Random Forest model's feature importance scores, geography stands out clearly: the location categories "Other Locations" and "Bangalore" together account for nearly 39% of everything the model uses to make its decision, with Product_ID contributing another ~19% and the delivery mode close behind. Individual sales agents and customer-referral leads also carry real weight, and the engineered date features show a smaller but non-zero contribution, justifying the effort of building them.

9. What This Means for FicZon

- Lead scoring can move from a monthly post-mortem to a real-time signal agents see the moment a lead lands, letting effort go where conversion is most likely.
- Because location is such a strong predictor, conversion likelihood clearly varies by region — a natural basis for territory-specific sales strategy.
- Lead source and delivery mode matter enough that marketing spend could be rebalanced toward the channels that produce higher-quality leads.
- Uneven workload across the 13 agents, and the fact that some agents' identities are themselves predictive, suggests that best practices from top performers could be identified and shared.

10. Honest Limitations and Next Steps

72% accuracy is a solid, usable starting point, not a finished product — roughly three out of ten leads will still be miscategorized, so this is best used to prioritize outreach order rather than to auto-reject leads outright. The report itself recommends validating the Status → Lead_Category mapping with FicZon's actual sales team, standardizing how status is entered in the future (to prevent a repeat of the CONVERTED/converted mismatch), monitoring the model's accuracy over time, and retraining periodically as products, agents, and seasonal patterns shift.

11. The Big Picture

Beyond the accuracy numbers, the most instructive part of this project is the target-mapping bug itself: a small, silent mismatch in text casing was enough to collapse an entire dataset into a single class, and it produced no error at the point it was introduced — only confusing downstream symptoms. Catching it required going back to first principles: inspect the real values, don't assume them. That discipline, more than any single model choice, is what made the rest of the analysis trustworthy.